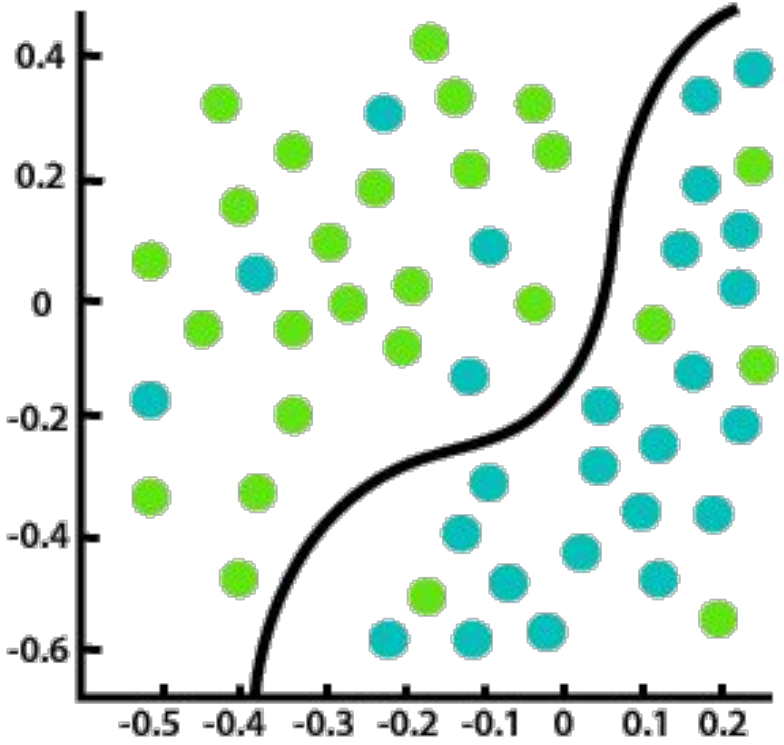
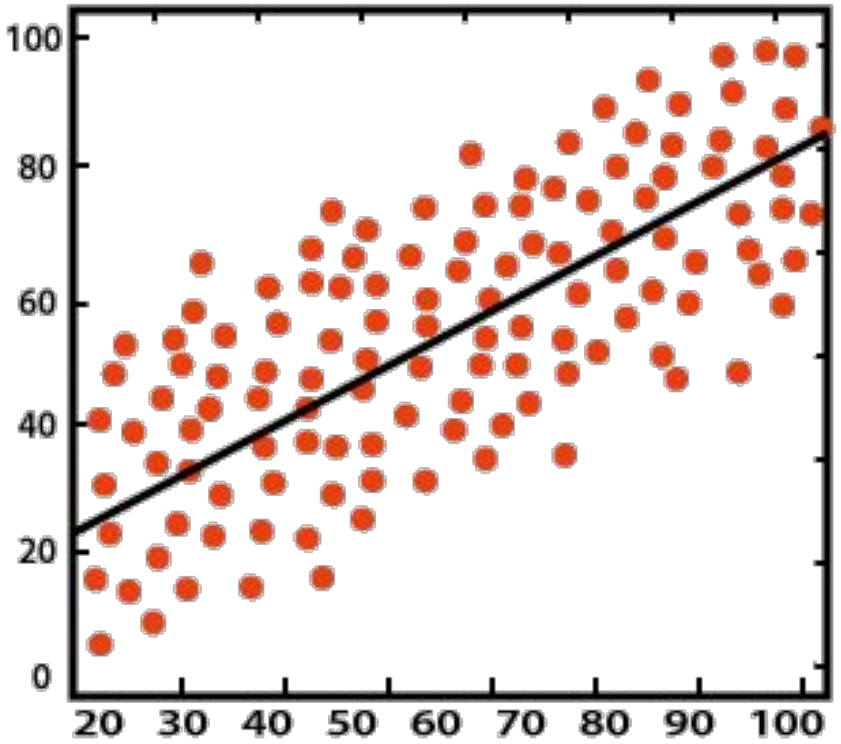
Bias-Variance Tradeoff, Regularization



Classification



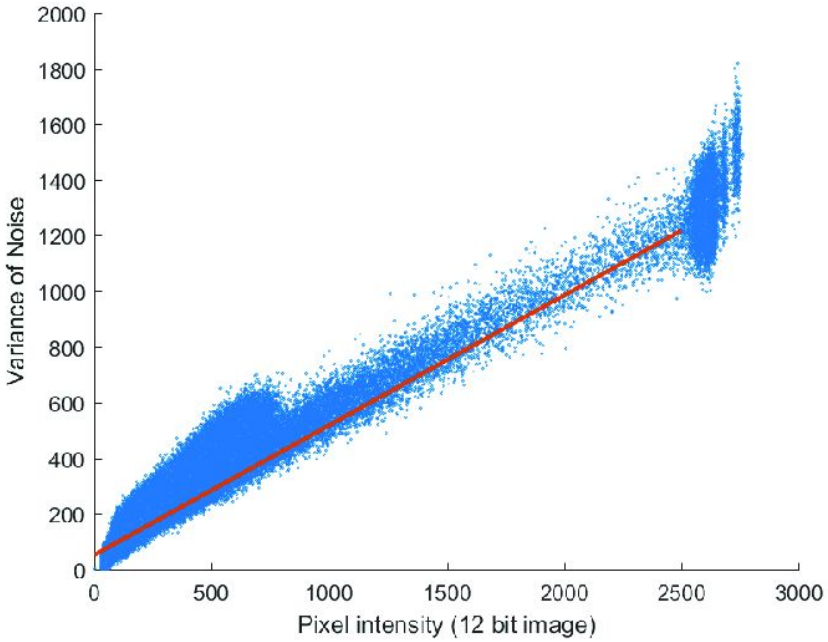
Regression

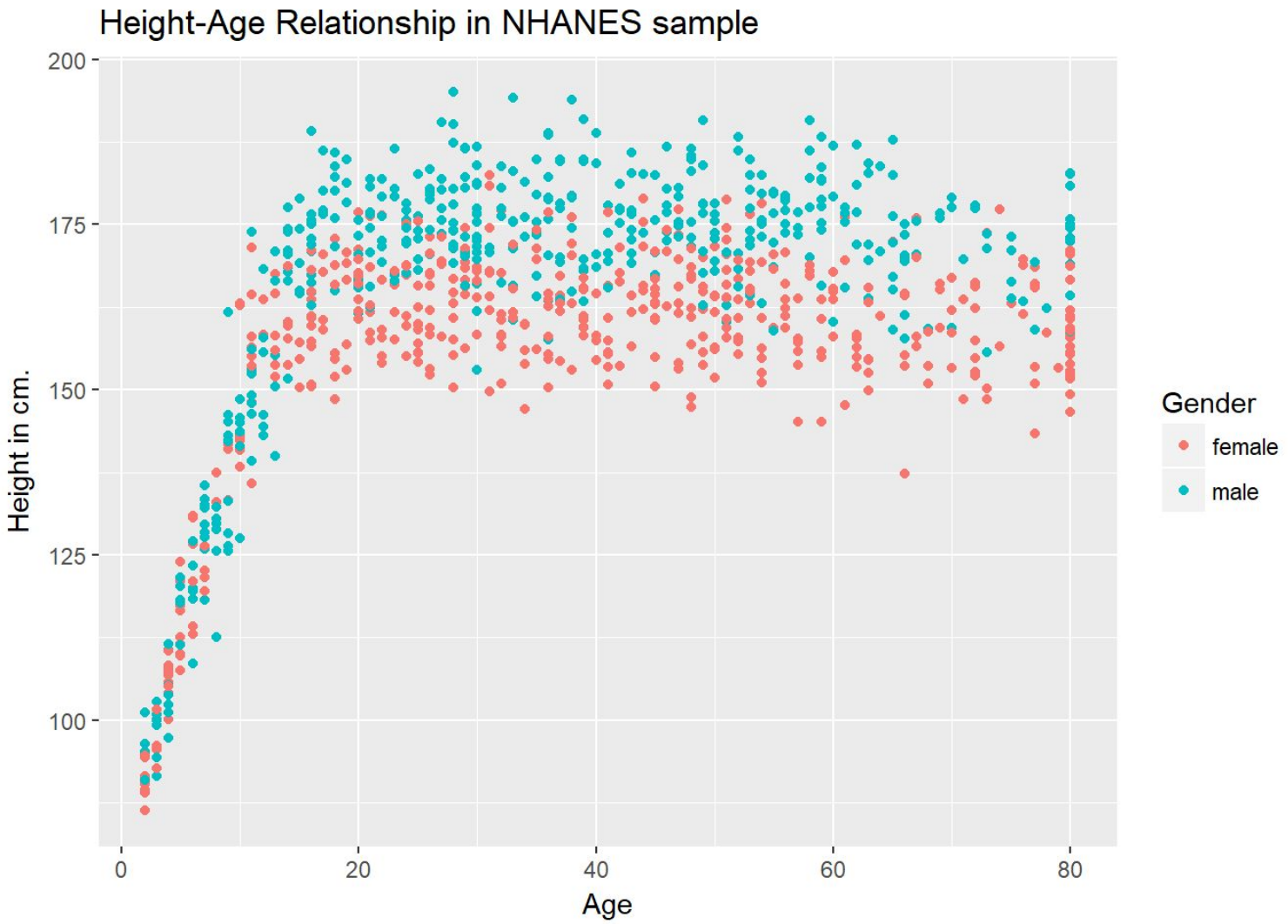
Real output = pattern + noise

$y = f(\mathbf{X})+\varepsilon$

f - pattern we're trying to learn

It could be linear regression, could be decision tree, etc.




Height-Age Relationship in NHANES sample
200
175
150
125
100
Height in cm.
0
20
40
60
80
Age
Gender
female
male

What we know about noise

ε - is noise

It could come from any kind of distribution

μ_ε - is generally 0.

Var(ε) = σ^2 (a non zero value)

What is expected value?

Expected value is always the average value.

Since the mean value of ε is 0, we say that expected value of ε is 0 (E(ε)=0)

While building a model we're assuming that σ doesn't change (but in real world it could change)

Let’s try to build a model

Looking at the data we might assume that the data is coming from a polynomial distribution but actually it might come from a sinusoidal function. We don’t know.

y = f(**X**)+ε

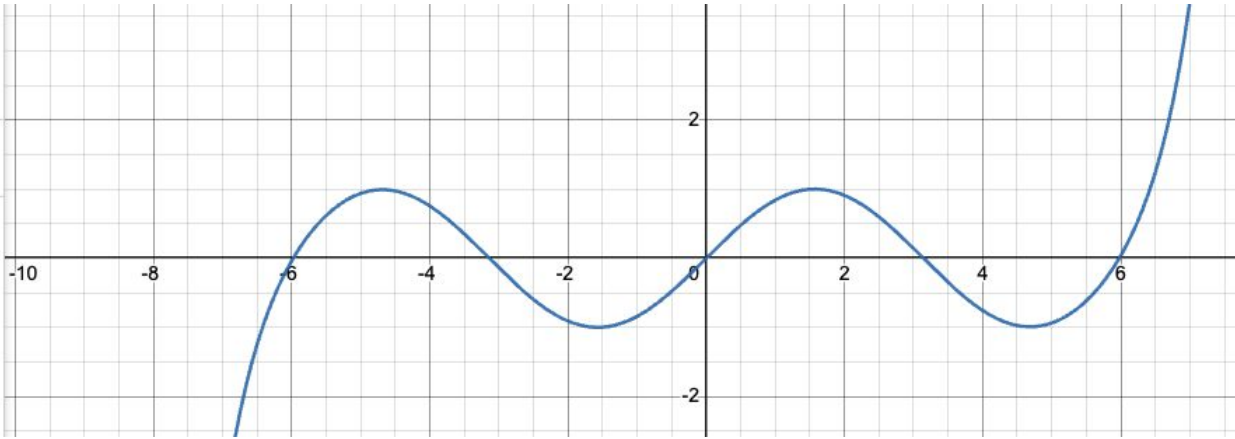
ε - is **irreducible error.** We can’t change it.

The model we build is y’ = f’(**X**)

Let’s assume the real distribution is coming from sinusoidal

Sin(x) = $$x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \frac{x^{11}}{11!} + \frac{x^{13}}{13!} \cdots$$

$$x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \frac{x^{11}}{11!} + \frac{x^{13}}{13!}$$



If we try to fit the data to a polynomial regression model, we will underfit.

After fitting the model using the training data, once we test our data on the training dataset, we are going to be left with residuals

It’s important to have no patterns in residuals

Residuals should represent the noise

Overfitting and underfitting in terms of patterns

Fitting sigmoidal function to a polynomial function our residuals will show patterns.

Basically, underfitting is taking patterns as noises

Overfitting is the opposite, taking noise as the pattern.

True underlying y=f(**X**) doesn't change, but y'=f'(**X**) is modeled with training data, and different version of the training data will have different f'(**X**) as long as we don't know the real f(**X**)

Low variance model vs High variance model

Let's assume we decided our data comes from a linear model

$y'_1 = f'_1(\mathbf{X_1}) \rightarrow y'_1 = w_0 + w_1x_1$

$y'_2 = f'_2(\mathbf{X_2}) \rightarrow y'_2 = w_3 + w_4x_1$

If w_0 is very close to w_3 and w_1 is very close to w_4, we call the models low variance models

If we get too different coefficients when using two different datasets as training sets, we call the models high variance models

Low bias vs High bias

We can have infinitely many f’(x) functions based on different training data.

Average of infinitely many f’(x) is E[f’(x)] - expected value of f’(x)

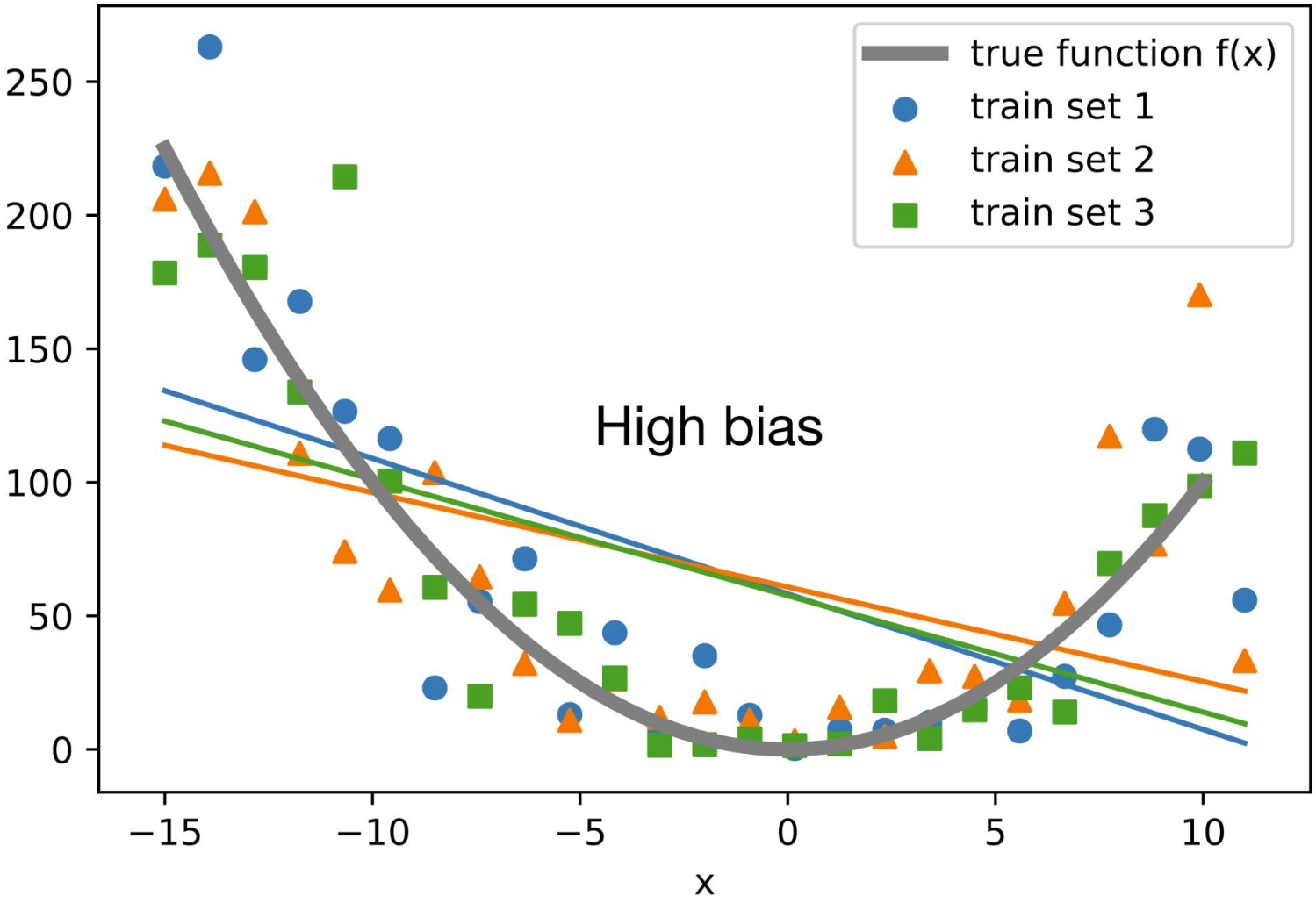
Bias = f_{true} - E[f’]

If E[f’(**X**)] = f(x) we call the model unbiased model

If it is closer to 0, it is a low bias model

$f'(x) = 1$

Is this a low bias or low variance?


true function f(x)
train set 1
train set 2
train set 3
High bias
250
200
150
100
50
0
−15
−10
−5
0
5
10
x

Bias vs Variance

Assume we have data, and we're trying to build a model with hyper-parameters

Let's assume we think of the model as polynomial and decide it has degree of d

$y = w_0 + w_1x + w_2x^2 + .. + w_dx^d + \varepsilon$

Polynomial models are flexible, and if the underlying model is wavy, the polynomial model can pick it up

Model degree means model complexity

Visualizing polynomial functions

https://www.desmos.com/calculator

Bias -> $f - E[f']$

MSE = $(y - f'(x))^2$

Expectation is average of SE of infinitely many data points, which is $E[(y-f'(x))^2]$ which we call **Test Error.**

Now let's see where bias-variance tradeoff happens in terms of MSE

MSE

$y = f + \varepsilon$ (true function + irreducible error)

$E[(y-f'(x))^2] = E[\ ((f+\varepsilon) - f')^2\ ] = E[\ ((f - f') + \varepsilon\ )^2\ ] = E[\ \ (f-f')^2 - 2\varepsilon(f-f') + \varepsilon^2\ ]$

Expectation is a linear operator, so sum of averages = average of sums

$E[(y-f'(x))^2] = E[(f-f')^2] - 2*E[\varepsilon*(f-f')] + E[\varepsilon^2]$

We know that $Var(\varepsilon) = \sigma^2$

$$\sigma^2 = \frac{\Sigma(xi - \bar{x})^2}{N}$$

We also know that variance is also $E[\varepsilon^2] - (E[\varepsilon])^2$

How?

Expected Value (Mean) of the Random Variable $E(X) = \Sigma\ [x * P(X = x)]$

$\text{Error} = X - E(X)$

$\text{Square of Error} = (X - E(X))^2$

$Var(X) = E[(X - E(X))^2]$

$Var(X) = E[X^2 - 2XE(X) + (E(X))^2] = E[X^2] - 2E[XE(X)] + E[(E(X))^2]$

$Var(X) = E[X^2] - 2E(X)E(X) + (E(X))^2 = E[X^2] -2(E(X))^2 + (E(X))^2 = E[X^2] - (E[X])^2$

Now, we're assuming that $E[\varepsilon] = 0$, so $Var(\varepsilon) = E[\varepsilon^2]$

If X and Y are independent, $E[X*Y] = E[X] * E[Y]$

Since we think of ε as a noise, and independent of f,

$E[(y-f'(x))^2] = E[(f-f')^2] - 2*E[\varepsilon*(f-f')] + E[\varepsilon^2]$ becomes

$E[(y-f'(x))^2] = E[(f-f')^2] - 2 * E[\varepsilon] * E[(f-f')] + E[\varepsilon^2]$, and we know that $E[\varepsilon] = 0$

So, $E[(y-f'(x))^2] = E[(f-f')^2] + E[\varepsilon^2]$

$= E[f^2 - 2ff' + f'^2] + \sigma^2$

$= E[f^2] - 2E[ff'] + E[f'^2] + \sigma^2$

f is the real function, which is constant. Expected value of constant is constant.

$= f^2 - 2f\, E[f'] + \mathbf{E[f'^2] + \sigma^2}$

$= f^2 - 2f\, E[f'] + (E[f']^2 - E[f']^2) + \mathbf{E[f'^2] + \sigma^2}$

$= (f - E[f'])^2 + E[f'^2] - E[f']^2 + \sigma^2$

True Test Error = $(\text{Bias}(f'))^2 + \text{Var}(f') + \sigma^2$

True Test Error = $(\text{Bias}(f'))^2 + \text{Var}(f') + \sigma^2$

Bias is squared, so it is positive

Variance is a non-negative number as well

This means, your true test error is always >= σ^2

What we can do is minimize $(\text{Bias}(f'))^2 + \text{Var}(f')$

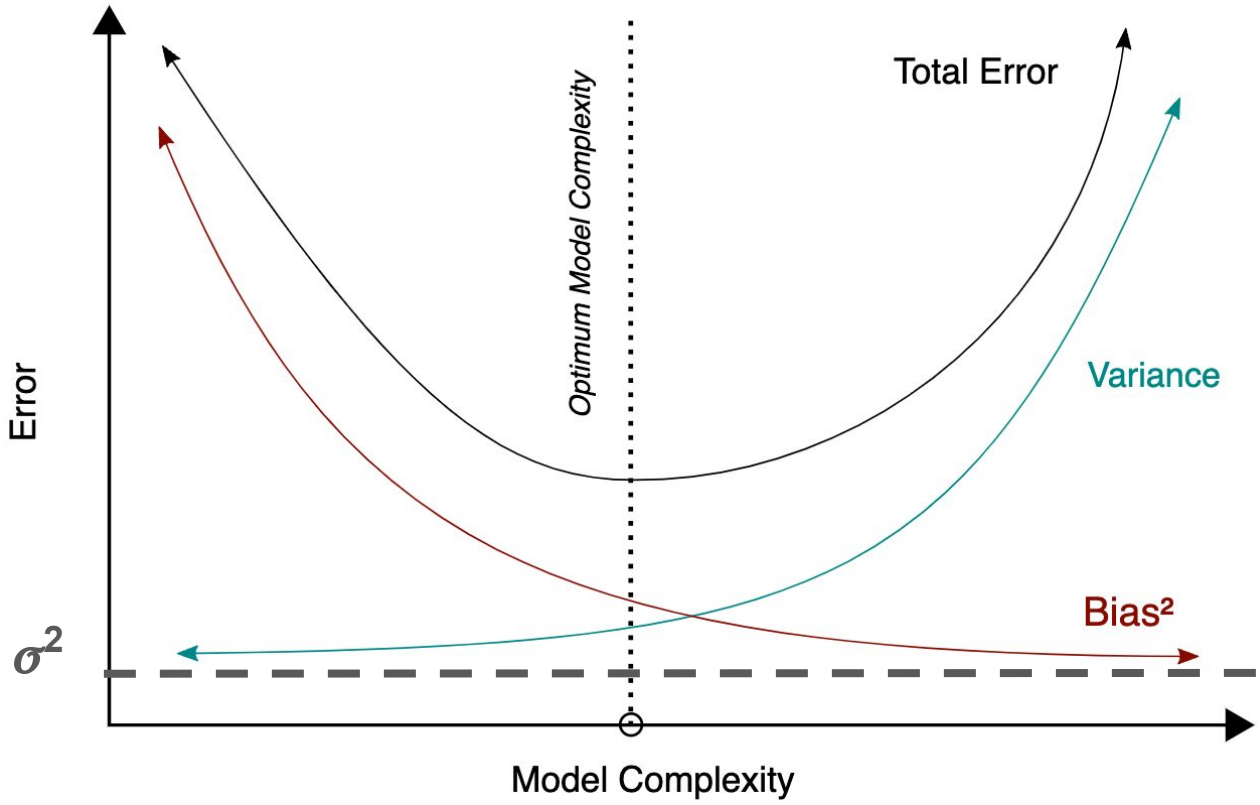
For ex : f'(x) = 1 is low variance, but our bias will spike up

Complex polynomial function could be low bias, but variance will go up

There are many methods (Sample Variance, Residual Variance in Regression, ML Models - Validation dataset, Bootstrap Resampling, Bayesian methods etc.) to calculate variance of noise to estimate the minimum test error

Bias vs Variance

Test Error = Bias^2 + Var + σ^2



Regularization

$f' = w_0 + w_1x + w_2x^2 + \ldots + w_dx^d$

As the degree of the model increases, the complexity increases, and as the complexity increases, the variance increases.

For each different training dataset the **w** will vary very wildly

To avoid this we either have to keep the model as simple as possible

Or, we regularize!

Regularization

To learn a model we minimize the Loss function.

Zero loss in high dimensional and high degree models means overfitting

We put an artificial break to the Loss function.

For ex: $\sum \mathbf{w}^2 \leq 100$

Lagrange multipliers

- Used to handle constrained optimization problems

Lagrange's method introduces a set of Lagrange multipliers, denoted as λ (lambda), for each constraint. You create a new function called the Lagrangian, denoted as L(x, λ), which combines the objective function and the constraints using the Lagrange multipliers:

$$L(x, \lambda) = f(x) + \lambda * g(x)$$

λ - Lagrange multiplier - in our case hyperparameter

Now our loss function becomes

$$L(x, \lambda) = L(\mathbf{w}) + \lambda * \sum \mathbf{w}^2$$

If λ is too high, the model will try to keep **w** very small

If λ is too low (zero), L(**w**) will tend to go to zero, and we will overfit

λ * ∑w² or **λ * ∑|w| -** We call them penalty, and it is always positive value

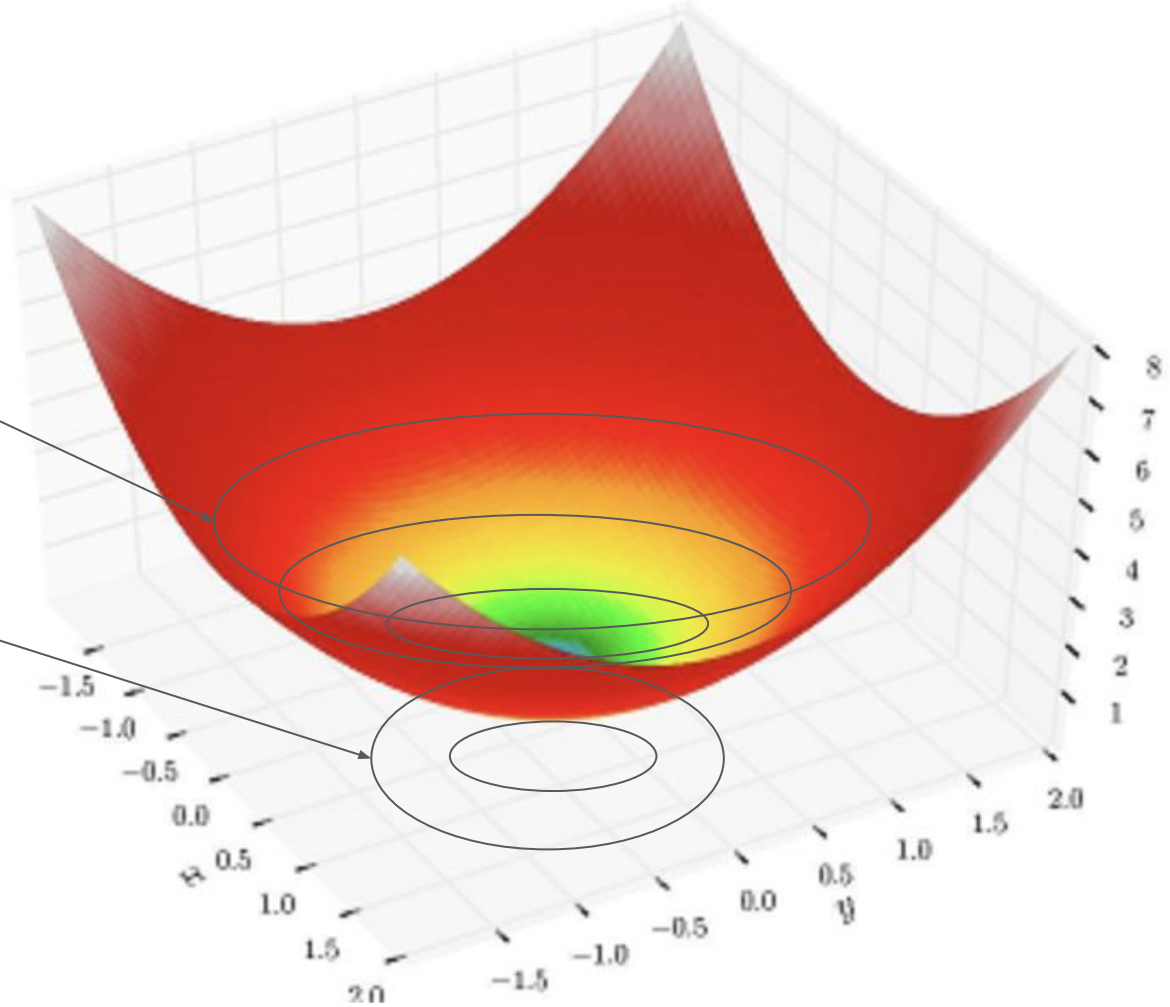
L2 - Ridge Regularization : $L(x, \lambda) = L(\mathbf{w}) + \lambda * \sum \mathbf{w}^2$

L1 - Lasso Regularization : $L(x, \lambda) = L(\mathbf{w}) + \lambda * \sum |\mathbf{w}|$

Loss

Level sets

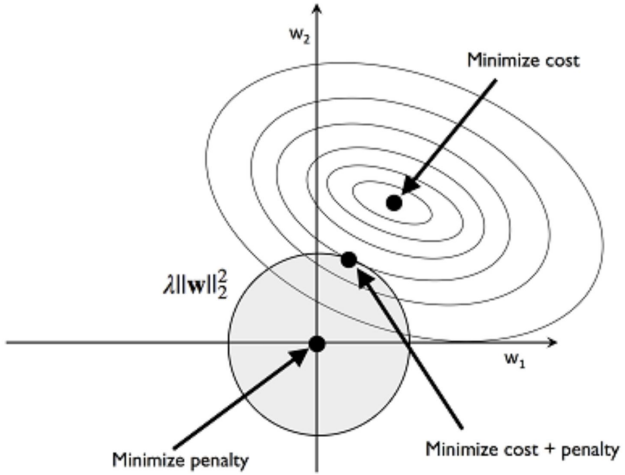
Contour plots



L2 vs L1

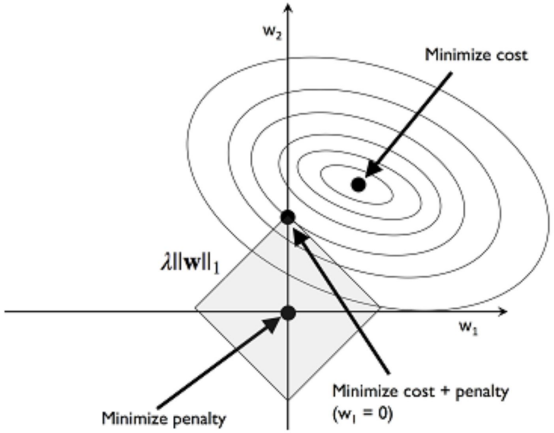
L2 - Ridge (smooth out, round) :

$w_0^2 + w_1^2 \leq 1$



L1 - Lasso (Least Absolute Shrinkage and Selection Operator):

$|w_0| + |w_1| \leq 1$



L1 vs L2 in terms of computation

Which one of the regularization method do you think is more computationally intensive?

Code

https://colab.research.google.com/drive/1RS8WYTykYsGNGf_7aVhtmKBbdIWcs46w?usp=sharing

https://colab.research.google.com/drive/1I4cLwT2IrEyCVt_r6GWRzwT-z7Bl4mkx?usp=sharing

ElasticNet

$$L(x, \lambda) = L(\mathbf{w}) + \lambda_1 * \sum \mathbf{w}^2 + \lambda_2 * \sum |\mathbf{w}|$$

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.ElasticNet.html

- **Handles Multicollinearity** : Elastic Net is particularly useful when dealing with multicollinearity, a situation where two or more predictor variables are highly correlated. Lasso tends to arbitrarily choose one of the correlated variables and ignore the others, while Ridge will shrink all of them roughly equally. Elastic Net, with its combined L1 and L2 regularization, can handle multicollinearity by selecting a group of correlated variables together (with non-zero coefficients) while also shrinking the coefficients of less important variables.

ElasticNet

- **Improved Generalization:** In cases where both L1 and L2 regularization are beneficial, Elastic Net can outperform Lasso or Ridge individually by combining their strengths.
- **Suitable for High-Dimensional Data**: Elastic Net is well-suited for high-dimensional datasets where there are many predictor variables relative to the number of observations. It can handle situations where you have more features than data points by promoting sparsity in the model.
- **Balanced Regularization:** Elastic Net allows you to strike a balance between L1 and L2 regularization through the alpha hyperparameter. When alpha is set to 0, it's equivalent to Ridge regression, and when alpha is set to 1, it's equivalent to Lasso regression. Adjusting alpha lets you control the degree of sparsity in the model and the emphasis on feature selection versus coefficient shrinkage.